

```

1 // Variables partagées [cite: 101]
2 int turn; // Indique à qui est le tour [cite: 102, 103]
3 boolean flag[2]; // Indique si le processus est prêt à entrer [cite: 102, 104]
4
5 // Structure pour le processus i (j est l'autre) [cite: 105]
6 do {
7     flag[i] = true; // Je suis prêt [cite: 107]
8     turn = j; // Je cède poliment le tour à l'autre [cite: 108]
9
10    // J'attends SI l'autre est prêt ET que c'est son tour [cite: 109, 110]
11    while (flag[j] && turn == j);
12
13    /* SECTION CRITIQUE */ // [cite: 111]
14
15    flag[i] = false; // J'ai fini, je ne suis plus prêt [cite: 112]
16
17    /* REMAINDER SECTION */ // [cite: 113]
18 } while (true); // [cite: 114]

```

```

1 // Variables partagées
2 boolean waiting[n]; // Initialisé à false pour les n processus
3 boolean lock = false;
4
5 // Structure pour le processus i
6 do {
7     waiting[i] = true; // [cite: 199]
8     key = true; // [cite: 200]
9
10    // Attente active : on tourne tant qu'on attend ET que le verrou est pris
11    while (waiting[i] && key) { // [cite: 202]
12        key = test_and_set(&lock); // [cite: 203]
13    }
14
15    waiting[i] = false; // [cite: 204]
16
17    /* --- SECTION CRITIQUE --- */ // [cite: 205]
18
19    // --- EXIT SECTION : La magie "Sans Famine" ---
20    j = (i + 1) % n; // [cite: 206]
21
22    // Chercher le prochain processus P[j] qui attend dans le cercle
23    while ((j != i) && !waiting[j]) { // [cite: 208]
24        j = (j + 1) % n; // [cite: 209]

```

```

25     }
26
27
28     if (j == i) { // [cite: 210]
29         // Si personne d'autre n'attend, on libère le verrou
30         lock = false; // [cite: 211]
31     } else { // [cite: 212]
32         // Sinon, on réveille le prochain processus en attente (P[j])
33         // SANS libérer le verrou (on lui passe directement la clé !)
34         waiting[j] = false; //
35     }
36
37     /* --- REMAINDER SECTION --- */
38
39 } while (true); // [cite: 214]

```

Pourquoi c'est la version "Sans Famine" (pour tes notes) : Au lieu de relâcher le verrou dans la nature (ce qui pourrait laisser un processus rapide le reprendre éternellement), le processus qui sort de la section critique scanne les autres processus (de $i+1$ jusqu'à revenir à i). S'il trouve un processus qui attend, il lui donne l'accès directement en changeant son `waiting[j]` à `false`, sans même relâcher le lock. Tout le monde est garanti d'avoir son tour au maximum après $N-1$ passages

```

1  do { C
2      while(test_and_set(&lock)); // Attente active (Spinning)
3
4      /* critical section */
5
6      lock = false;
7
8      /* remainder section */
9  } while (true);

```

```

1  do { C
2      // Tente de passer lock de 0 à 1.
3      // Tant que lock != 0 (donc occupé), la boucle tourne.
4      while (compare_and_swap(&lock, 0, 1) != 0);
5
6      /* critical section */
7
8      lock = 0; // Libération
9
10     /* remainder section */
11 } while (true);

```